Does human review intent help a coding agent? A three-arm replay on merged pull requests

> **Status: draft with final results (2026-11-07).** All 56 runnable PRs are complete. One item marked [check] waits on the author's spot-check of two PRs.

Zili (Allen) Luo · Independent study, November 2026 · Not peer reviewed · Draft

Abstract

Tests written alongside a fix check the fix's own choices, such as names and structure, as well as the requested behaviour. Human code review often says what a change should do and why. This study asks whether that intent, given to a coding agent before it writes anything, moves its fix closer to what the project accepted. A sample of merged pull requests (PRs) from the AgentReviewChat dataset is replayed from the commit just before it, three times, with Claude Sonnet 5.5 in Claude Code: with the linked issue only (A), with the issue plus a summary of the human review (B), and with the issue plus the same kind of summary of AI-agent review (C). Results are scored by the PR's own fail-to-pass tests and by a behaviour rubric written from the issue and the merged PR, never from the review, and checked by two blind model inspectors. The analysis plan was frozen before any agent run. On 56 PRs from 22 repositories (29 Python, 27 Go), the mean rubric score was 0.47 in arm A, 0.63 in arm B and 0.56 in arm C. Arm B scored higher than arm A by +0.16 (95% bootstrap CI +0.07 to +0.25; Holm-adjusted p = 0.003), the pre-registered "Signal" verdict, and the result holds with the second inspector and with repository-clustered intervals. Arm B against arm C differed by +0.07 (CI -0.02 to +0.15; adjusted p = 0.15): no large effect. Few runs passed the PR's own tests (5, 8 and 8 of 56): in 87 of 147 unresolved runs the tests did not compile or import against the agent's code, typically because they call names only the merged PR introduced. The median B run used 51% more tokens than the median A run.

1. Introduction

When a project reviews a pull request, reviewers often state what the change is meant to do and why. That information is rarely in the issue, and an agent that works from the issue alone has to guess it. The usual check on an agent's patch does not help here: tests confirm that the code does what its author built, which is not the same as what was asked. Human review is one of the few places where the "what and why" is written down by someone who knows the project.

This study asks one question. Does the intent in human code review (what the change should do and why, with code removed), given to a coding agent up front, move the agent's fix closer to what the project accepted? Two comparisons make the question precise. B against A asks whether review intent beats the issue alone. B against C asks whether human review intent differs from the same

kind of summary made from AI-agent review. Arm C also controls for length: if B beats A only because it carries more text, C should beat A too.

The contribution is a three-arm replay design on real merged PRs, a scoring scheme that never builds its checklist from the review text, and a result, whichever way it comes out. A negative result is a result. The study is small and pre-registered: the plan was committed to a public repository before the pilot's first agent run, later changes are listed as dated amendments, and every model call ran on a personal Claude subscription rather than a funded API budget. It supports claims about these 22 repositories, these two languages and this agent, and no wider claims.

2. Related work

The nearest published work found is c-CRAB, a code review agent benchmark (Zhang et al., arXiv 2603.23448). Review tools comment on a PR's patch, and Claude Code (Sonnet 4.6) then revises the patch, starting from the PR head. Hidden tests written from the human review score the revision. Revisions guided by the tools pass 20.1-32.1% of the tests (Table 5), and the human review's own fix passes all of them, which follows from how the tests were validated. This study differs in three ways: it starts from the base commit, not from an existing patch; review is given as intent before any code is written, not as input to a revision; and it has an issue-only arm.

The closest work on what an agent is told up front is "What Makes a Good Bug Report for an AI Agent?" (Khatib et al., arXiv 2607.07593). It removes parts of the problem statement (suggested fixes, localization, expected behaviour) on SWE-bench Pro and measures the effect. That content comes from the issue, not from review, and it has no human-versus-AI comparison. "Intention is All You Need" (Guo et al., arXiv 2502.08172) extracts a reviewer's intent from review comments, but uses it to refine existing code rather than to write the change from the base commit.

SWE-Review (Wang et al., arXiv 2607.06065) has a reviewer agent judge an AI-generated patch on SWE-bench Verified and send feedback for one revision. Its Table 1(b) compares no review, a decision only, a teacher review and an oracle review (3%, 8%, 21% and 32% resolved after revision, on 100 instances). It covers AI review only, and only after a patch exists. The closest precedent for "human discussion helps" is a pair of baseline rows in MASAI's Table 1 (Arora et al., arXiv 2406.11638): OpenDevin resolves 25.00% of SWE-bench Lite with the benchmark's `hints_text` and 16.00% without. These rows appear to be different OpenDevin versions, so the gap is not a clean ablation. Those hints are issue-thread comments, not review.

Leakage is a standing risk when human text is given to an agent. SWE-Bench+ (Aleithan et al., arXiv 2410.06992) reports that in 32.67% of SWE-Agent + GPT-4's successful patches the solution was given directly in the issue report or its comments. This study answers with a leak check on every summary (Section 4.2) and a rubric that avoids required names and structure.

Work on missing intent in coding tasks is related background: Ambig-SWE (Vijayvargiya et al., arXiv 2502.13069; accepted at ICLR 2026), "Ask or Assume?" (Edwards and Schuster, arXiv 2603.26233) and CLARITI (Vijayvargiya et al., arXiv 2604.14624). They study an agent asking for what an underspecified task leaves out. Here the missing information is supplied up front, from

review. For scoring, Shi et al. (arXiv 2511.10865) have an LLM draft a rubric per bug, which developers then refine. They report that at k=20, 96% of patches passed reproduction tests while 80% were judged valid by the rubric-based LLM judge, and that of 504 patches that passed reproduction tests, the judge rated 219 (43.5%) invalid. The rubrics here are model-drafted and not human-refined, which Section 7 states as a limit.

The data come from "Human-AI Synergy in Agentic Code Review" (Zhong et al., arXiv 2603.15911 v2), whose dataset is on Hugging Face as AgentReviewChat (`Suzhen/AgentReviewChat`, MIT): 278,790 inline review conversations on GitHub PRs, each labelled by whether a human or an AI review agent made the first comment. That paper is observational. A search on 2026-11-07 found no published study with this three-arm design (issue; issue plus human-review intent; issue plus agent-review intent) on merged PRs replayed from the base commit. The search was short and read abstracts, so "not found" is the claim.

3. Research questions and hypotheses

The primary outcome is the rubric score from the Opus inspector (Section 4.5).

- **H1.** Rubric scores in arm B differ from arm A.
- **H2.** Rubric scores in arm B differ from arm C.

Both are two-sided. Secondary questions are reported but cannot change a verdict: the stage-1 resolved rate, C against A, inspector agreement, and token efficiency.

4. Method

4.1 Data and selection

The pool is the merged PRs in AgentReviewChat with at least one human-started and at least one agent-started review conversation: 15,409 PRs in 277 repositories. "Human-started" and "agent-started" follow the dataset paper's rule (the first commenter of the conversation), which reproduces that paper's Table I. Agents are the 16 accounts in the paper's list.

Languages are Python and Go, by each repository's primary language on GitHub. A PR is eligible if it (1) links at least one closing issue; (2) changes at least one runnable test file (Python `test_*.py` or `*_test.py`; Go `*_test.go`); (3) has a commit pushed after the first human review, so the review could have changed the code; and (4) changes at most 100 files and has a merge commit. One GitHub pass over every Python and Go pool PR found 238 eligible PRs in 16 repositories for Python and 125 in 16 for Go. Four repositories were excluded because their environments are too heavy or impossible to build here: ray-project/ray, lablup/backend.ai, ansys/pymapdl (it needs licensed ANSYS) and apache/tvm.

The draw used seed 20261107. Eligible PRs are shuffled within each language; walking the shuffled list, a PR is taken if its repository has fewer than six taken, until 35 per language. Pilot PRs are removed first. The draw took 70 PRs from 22 repositories. There are no replacements: a drawn PR

that later fails a build check is excluded and counted. The build checks, in order, are: the merge commit's diff against its first parent has the same file set as the PR; the project environment builds at the base commit; at least one fail-to-pass test exists; and both summaries exist. The plan expected 50-60 usable PRs.

4.2 Arms and summaries

All three arms run on the same PR, from the same base commit, with the same model, tools, budget and prompt. They differ in one block of text. A gets the issue (title and body of each closing issue, fetched at build time; issue comments are not included). B adds a summary of the human review. C adds a summary of the AI-agent review. The block has the same header in B and C and does not say where it came from.

The review material comes from AgentReviewChat only, so it is inline review conversations. For B it is every comment in a human-started conversation written by a human other than the PR author. For C it is every comment in an agent-started conversation written by one of the 16 agent accounts. Each comment comes with its file path and diff hunk, and GitHub suggestion blocks are removed before summarising.

One summariser call per side per PR uses the same prompt for both sides: Opus 5.5, medium effort, no tools, at most 150 words, saying what the change should do and why, never how, with no code. A leak check then runs. A summary leaks if it contains (a) a name added by the merged PR, meaning an identifier in an added line of the merged diff that appears neither in the base versions of the touched files nor in the issue, or (b) six or more consecutive code tokens from an added line of the merged diff. A leaking summary is regenerated once with the flagged names listed as forbidden. If it still leaks, each flagged name is replaced with "[a new name]". The procedure is the same on both sides, and hits are counted per arm.

Amendment 1 (2026-11-07, before any agent run). Reading the first pilot PR's two summaries by hand (home-assistant/core #148639) found one that quoted a code fragment from the draft, which the leak check does not catch, and one at 159 words, over the cap. Both broke rules already in the plan, so the plan enforced them rather than changed them. The prompt now asks for requirements for the finished change, not edits to a draft the developer never saw, and forbids backticks. An automatic check for code spans and length folds into the one regeneration. If the second answer still breaks a rule, backticks are removed and the text is cut at the last whole bullet or sentence within 150 words. Both sides get the same procedure, and first-answer rule breaks, regenerations and cuts are counted per arm.

4.3 Agent setup

The agent is Sonnet 5.5 (`claude-sonnet-5-5`) at medium effort, run through `claude -p` on a Claude subscription. Model, flags and Claude Code version are logged per run. Tools are Read, Edit, Write, Glob, Grep and one MCP tool, `run`, which executes a shell command inside the PR's Docker container. The container has no network. There is no host shell, no web tool and no subagent.

Claude Code runs without user, project or local settings; the user-level CLAUDE.md is excluded and auto-memory is off. Project instruction files inside the checked-out repository load as they would for any developer. The checkout sits in a neutral folder named after the repository, with a fresh one-commit git history, so later commits do not exist anywhere the agent can reach. The prompt is the same in every arm, with the review block present only in B and C. It asks the agent to resolve the issue, to use `run` for commands, not to commit, and not to refer to the task or its instructions in code, comments or docs.

The budget is one run per arm per PR with 45 minutes of wall clock. A run that hits the time limit is scored on the diff it left. The three arms of a PR run back to back in an order shuffled per PR with seed 20261107. A run cut off by a usage limit, or one that fails for an infrastructure reason, is discarded and rerun, up to two times. A model refusal is scored as it stands. A PR without a valid run in all three arms leaves every comparison and is counted. Every call logs its usage (input, cache and output tokens, turns, duration) by role and model.

4.4 Stage 1: tests

Before scoring, added comment lines that mention review, reviewer, guidance or feedback are removed from the agent's diff in every arm. Scrubs are counted per arm.

Test files are the PR's changed files that match the language's test paths. For Python these sit under `tests/`, `test/` or `testing/`, or match `test_*.py`, `*_test.py` or `conftest.py`; for Go they match `*_test.go` or sit under `testdata/`. The tests run are the runnable changed test files (Python) or, by Amendment 2, the test functions defined in the changed `_test.go` files, inside their packages (Go). Fail-to-pass (F2P) tests fail, error, or do not compile or collect at the base commit plus the PR's test files, and pass at the merge commit. The merge commit runs twice, and a test whose result differs between runs is dropped. Pass-to-pass (P2P) tests are run tests that pass at both. To score an agent, the PR's test files are copied over its tree and the same tests run. A run is resolved if every F2P test passes and every P2P test still passes.

4.5 Stage 2: rubric and inspectors

The rubric is never built from review text. B was given those points, so a checklist drawn from them would favour B by construction. A rubric writer (Opus 5.5, medium effort, no tools, once per PR) sees the issue, the PR title and description, and the merged diff, and never sees review text or any agent output. It writes 3-8 behaviour items that a different correct implementation could also meet, with no required names, file locations or code structure unless the issue asks for them. Tests are not rubric items.

Two inspectors (LLM judges) judge each implementation, each in one call with no tools. Opus 5.5 at medium effort gives the main score; Sonnet 5.5 at medium effort gives agreement and robustness. Each sees the issue, the rubric, the scrubbed diff, the full touched non-test files after the change and the stage-1 result, and returns met, not met or can't tell for each item, with file and line evidence. Inspectors are blind to the arm and to each other. There is no tie-breaker and no

averaging. The rubric score is the share of items met; can't tell counts as not met, and an empty diff scores 0 without an inspector call.

4.6 Analysis

The unit is a PR with all three arms scored. The primary outcome is the rubric score from the Opus inspector. H1 (B against A) and H2 (B against C) are each a two-sided Wilcoxon signed-rank test on the per-PR differences, with zero differences dropped (scipy's default), and Holm correction across the two at 0.05. Each comparison is reported with the mean paired difference and a 95% bootstrap interval (10,000 resamples of PRs, seed 20261107).

The verdict rules are fixed. Signal means adjusted p below 0.05 with B higher. Reversed means adjusted p below 0.05 with B lower, and it is reported as that. No large effect means adjusted p at or above 0.05; this is not "no effect". Inconclusive means fewer than 30 complete PRs. On power (the plan's estimate for 55 PRs): a standardised paired effect of about 0.4 is detected about 80% of the time at the Holm-adjusted level, and smaller effects often are not. A null rules out a large effect and says little about a modest one.

The following are reported and cannot change a verdict.

1. Stage 1: resolved rate per arm, with an exact McNemar test for B against A and B against C.
2. C against A, with the same test as the primary comparisons.
3. Inspector agreement: Cohen's kappa at item level between Opus and Sonnet (met against not met), and the 3-class kappa. H1 and H2 are rerun on Sonnet's scores; if a verdict changes, the report says the result depends on the inspector.
4. H1 and H2 bootstrap intervals with resampling clustered by repository.
5. Rubric score with can't-tell items left out of the denominator.
6. Token efficiency: per run, total tokens (input, cache read, cache write and output), turns and wall-clock time; the median per arm; a Wilcoxon signed-rank test for B against A and B against C. The question is whether up-front intent also makes the agent cheaper. It was fixed in the plan before any run because papers often report it.
7. Each result split by language, descriptive only.
8. Summary lengths, leak-check hits, scrubs, timeouts and refusals per arm.

Anything else is exploratory and labelled so. Rubrics and inspections were not validated against human labels. [check: the author's spot-check of two PRs is pending; update this sentence and Section 7 when it is done.]

4.7 Pilot

A pilot of ten PRs, not counted in the results and removed from the main draw, was drawn with seed 20261107 from the eligible PRs of four repositories: three from home-assistant/core, two from getsentry/sentry-python, three from gruntwork-io/terragrunt and two from zitadel/zitadel. It ran all three arms with Sonnet 5.5, and arm A once more with Opus 5.5 at medium effort as the agent. The pilot looked at whether each PR builds and has F2P tests, at plumbing (empty runs, crashes, timeouts, leak hits, scrubs), at every summary read by hand for how-level content, and at usage per

role and model, which set the main run's pace. For the ceiling check it looked at arm A alone, Sonnet against Opus. Scores for arms B and C were computed to test the pipeline but not printed or read, and no comparison between arms was made on pilot data. The ceiling rule said that if Sonnet in arm A resolved at least 8 of 10 PRs with a mean rubric score of at least 0.9, the selection would be tightened before the main draw. Nine of the ten pilot PRs got runs; zitadel #11009 got none because of an orchestration error (two processes ran it into one folder). The pilot found one harness fault: the prompt said the checkout "is mounted at /work", and in 21 of 36 pilot runs the agent passed /work paths to its file tools at least once. Those calls were denied, and several runs then stopped without changing anything. Amendment 3 changed the wording before any main-run agent run, the same in every arm, and a check run after the change had no denials. The ceiling rule did not fire: in arm A, Sonnet resolved 1 of 9 piloted PRs with a mean rubric score of 0.48, and Opus resolved 3 of 9 with 0.76. Both figures are depressed by the /work fault, so Section 5 also reports arm A's resolved rate against the rule. Usage per run set the main run's pace: Sonnet agent runs used about 2.2 million tokens over 27 calls, and the main run paused new calls when the five-hour usage window passed 80%.

5. Results

5.1 Selection and exclusions

Of the 70 drawn PRs, 14 were excluded at the build checks. Four failed check 1: the merge commit's file set differs from the PR's (all four from zeta-chain/node). One failed check 2 (environment): its tests need a live Cosmos DB database. Seven failed check 3 (no fail-to-pass test): for five of them the PR's tests ran and passed both before and after the change; one has only a browser screenshot test; and for one (#40615) the only tests that exercise the bug can run only against live services. Two failed check 4: arm B uses comments from humans other than the PR's author, and in these PRs the human-started conversations held only the author's own replies, so there was nothing to summarise. Appendix A lists each exclusion with its cause. Two PRs were repaired after a setup fault, as Appendix A describes: azure-cli #32298, whose tests ran past the 30-minute limit until the same tests were run in eight parallel workers, joined the run; azure-sdk-for-python #40615 moved from check 2 to check 3 once its recorded tests replayed offline. That left 56 complete PRs (29 Python, 27 Go) from 22 repositories (10 Python, 12 Go). No PR has an incomplete set of arms. No run hit a usage limit, timed out or was refused. Three runs ended with no code change (one in A, none in B, two in C; avalanchego #4147 C and mimir #12752 A and C) and score 0 by the plan's rule; without those two PRs, H1 is unchanged (+0.161 on 54 PRs, Wilcoxon $p = 0.002$; exploratory).

5.2 Primary: rubric score

	A	B	C
Mean rubric score (Opus inspector), n = 56	0.465	0.626	0.560

Comparison	Mean paired difference	95% bootstrap CI	Wilcoxon p (non-zero pairs)	Holm-adjusted p	Verdict
H1: B vs A	+0.160	+0.070 to +0.252	0.0015 (40)	0.0030	Signal (B higher)
H2: B vs C	+0.066	-0.015 to +0.147	0.154 (42)	0.154	No large effect

B scored higher than A on 27 PRs, lower on 13 and the same on 16. The median paired difference is zero: the mean gain is driven by a subset of PRs with large differences, and 13 PRs scored lower with B. Appendix C shows one PR, drawn at random, where B scored lowest. B scored higher than C on 27 PRs, lower on 15 and the same on 14. Arm A was far from the ceiling (mean 0.465, 5 of 56 resolved), so the ceiling rule from the pilot did not apply.

5.3 Stage 1: tests

	A	B	C
Resolved (all F2P pass, no P2P breaks)	5/56	8/56	8/56

Exact McNemar: B against A, 4 B-only and 1 A-only (p = 0.375); B against C, 3 B-only and 3 C-only (p = 1.0).

Most stage-1 failures are not about behaviour. In 87 of the 147 unresolved runs (31 of the 56 PRs), the PR's tests did not compile or import against the agent's code. They call names that only the merged PR introduced: unexported Go identifiers (`undefined: docURL`), private attributes (`_sync_credential`) and new constants (`CONF_SITE`). An agent that does not pick the same internal names fails every F2P test in that file, whatever its fix does. This happened at a similar rate in every arm: 30 of 51 unresolved runs in A, 28 of 48 in B and 29 of 48 in C. Stage 1 here is dominated by whether the agent used the project's internal names, which is why the plan made the rubric the primary outcome. Adapting the tests to the agent's names with a model was considered and not done: it would add a third model step that could bend the tests, for an outcome that cannot change a verdict.

5.4 Inspector agreement and robustness

The two inspectors agreed closely on the 1,164 rubric items: Cohen's kappa 0.861 with two classes (met vs not) and 0.814 with three. On Sonnet's scores, H1 is +0.189 (Holm-adjusted p = 0.0002, Signal) and H2 +0.057 (p = 0.157, no large effect), so neither verdict changed between the two judges. Both are Claude models and neither was checked against human labels, so this is agreement, not validity. Intervals clustered by repository are +0.075 to +0.236 for H1 and -0.014 to +0.142 for H2. With can't-tell items left out, the scores are A 0.469, B 0.633 and C 0.569 (B-A +0.164, B-C +0.064).

5.5 Token efficiency

Median per run	A	B	C	Wilcoxon B vs A	B vs C
Total tokens	85,488	129,164	110,858	$p < 0.001$	$p = 0.17$
Turns	12	16	14	$p = 0.002$	$p = 0.50$
Seconds	43	68	63	$p = 0.005$	$p = 0.34$

With human-review intent the agent did more: a median of 51% more tokens than with the issue alone, more turns and more time. B and C do not differ clearly.

5.6 Other reported items

C against A: +0.094 (bootstrap CI +0.010 to +0.183; Wilcoxon $p = 0.080$). Agent-review intent may also help, but the evidence is weaker than for human review, and this comparison is secondary: it cannot change a verdict.

By language (descriptive): in Go ($n = 27$), rubric scores were A 0.398, B 0.608, C 0.488; in Python ($n = 29$), A 0.528, B 0.642, C 0.626. B is above A in both languages. Descriptively, with no test and about 28 PRs per language, C was close to B in Python and lower in Go.

Summaries: the median length was 142 words for B and 140 for C. The leak check fired on 8 first answers in B and 6 in C. The rule check (code spans or length) fired on 32 first answers in B and 21 in C; after the one regeneration, 25 B and 16 C summaries were cut to 150 words. B summaries broke the rules and were cut more often than C summaries; both sides followed the same procedure. The comment scrub removed 2 lines, both in arm B. There were no timeouts and no refusals.

6. Discussion

H1 holds: on these 56 PRs, a summary of human review given before the agent starts raised its score on a model-written rubric of the merged PR's behaviour by 0.16 on average (Opus judge; 0.19 with the Sonnet judge). H2 does not: human-review intent was not shown to beat agent-review intent; the difference was 0.07 and its interval includes zero.

What each outcome would mean was stated before the data were read. If B beats A and B beats C, the intent in human review carries something that the issue and AI review do not, on these PRs and for this agent. If B beats A and C does too, with B and C close, the gain may come from any added statement of intent or from added text, and this design cannot separate the two. If B beats A but not C, human review is not shown to differ from agent review as a source of intent. If neither B nor C beats A, a summary of review given up front does not help this agent on these tasks, at a size this study can detect. If B scores lower than A, the summary may mislead or distract, and the paired differences will show which PRs drive it. In every case a null result rules out a large effect, not a modest one (Section 4.6).

The third case occurred: B beats A, and B is not shown to beat C. C's own gain over A (+0.09, reported only) sits between the two, so the data also fit the second case, where any statement of intent helps and its source matters less. A study with more PRs, or one that varies the AI review agent, could tell these apart. The gain is uneven: about half the PRs gain, nearly a third score the same in A and B, and about a quarter (13) score lower. Which PRs gain, and why, is not analysed here. The cost is real: the agent spent about half again as many tokens with human-review intent. Whether the gap between human and agent review is real but small, or depends on which review agent wrote the comments, is the clearest open question.

6.1 Planned follow-up: rubric items present before review

This follow-up is written down before it is run; its results, if run, will go in a separate follow-up report. The rubric is written from the final merged diff on purpose: it is what the project accepted, and a rubric built from the review text would grade arms B and C partly on their own guidance. But the merged diff also contains changes made after review, some of them at a reviewer's request, and arms B and C are told about those requests (Section 7). Reviews do not line up commit by commit, since a PR has human and agent conversations at different times. So the follow-up uses one cut point, the first review comment of either kind, rather than attributing each commit.

1. For each of the 56 PRs, one model call is given the rubric items and the PR's code at its last commit before the first review comment, and tags each item as already present there or not. A sample of tags is checked by hand.
2. No agent runs are repeated: the rubric score is recomputed from the existing Opus verdicts on the "present before review" items only, for each arm.
3. H1 and H2 are rerun on these scores with the same tests and Holm correction, and reported as exploratory. PRs with no such item are dropped and counted.

If B still beats A on the pre-review items, the gain is not only "the agent did what the reviewer asked". If the gain disappears, the main result mostly measures uptake of review requests, which is still a finding about review as guidance, but a narrower one.

6.2 Planned follow-up: an oracle arm (issue plus rubric)

A rubric is, in effect, a full description of the task: what a complete issue would have said. In the main study no arm sees it, because it is the answer key. A fourth arm, O, would give the agent the issue plus the rubric's behaviour items, on the same 56 PRs and with the same harness and inspectors. Arm O is not a fair comparison with A, B or C: it is scored against the text it was given. It is a calibration run with two uses.

1. Feasibility of the rubric. If arm O scores low, items are too strict or cannot be judged from the code, which bears on the measure's validity without human labels.
2. Scale of the effect. Arm O sets a practical ceiling, so B's gain can be stated as a share of the gap between issue-only and full description: (B - A) / (O - A).

Results would go in the follow-up report, labelled exploratory. A related but separate design would give every arm the full description and treat review as information about errors to avoid along the

way, measured by the defects reviewers caught; that is closer to c-CRAB (Section 2) and is left for future work.

7. Threats to validity

The rubric may favour arm B through the review itself. Eligibility requires a commit pushed after the first human review, so the merged PR, from which the rubric is written, contains changes the reviewers asked for, and arm B is told about those requests. Part of B's gain may therefore be "the agent did what the reviewer asked", which is close to what the rubric measures, even though the rubric writer never sees review text. A sensitivity analysis on rubric items already present before the first review would test this; it is planned (Section 6.1) and not done here. The same path applies to arm C wherever agent review was acted on. The summaries can also carry where to work, not only what and why: the leak check catches names new in the merged PR, not existing file or function names, and no main-run summaries were audited by hand. There is no length-matched or placebo text arm, so "intent helps" cannot be fully separated from "more task-relevant text helps". The study cannot show why intent helps or not. It measures closeness to what the project accepted, not code quality in general. The rubric comes from the merged PR, so "closer to what was accepted" is the outcome by definition, and a different good fix can still lose items. Rubrics and inspections are model-made and were not validated against human labels. The agent may have seen these repositories, or these PRs, in training. It may not affect the arms alike: a review summary could cue recall of the merged PR more than the issue alone does.

Review is summarised once by one model, and a different summariser may carry different intent. Only inline review conversations are in the data; top-level PR comments and discussion elsewhere are not. Review comments respond to a draft the agent never sees, so some intent arrives as advice about code that does not exist in the agent's run. Issues are fetched now and may have been edited after the PR. Each PR has one run per arm, so run-to-run noise is in the comparisons and is not measured separately. The study covers two languages, 22 repositories, and AI review from the agents in one dataset, and results may not carry to other projects or agents. The summary and leak rules reduce the risk that a summary hands the agent the answer, but they do not remove it, and Amendment 1 shows that the first rules missed a code fragment.

Operational events during the main run are disclosed here. Each language ran as one process, paused overnight by a planned stop and resumed in the morning. Three further restarts happened: the Go process after Windows briefly locked a work folder and two PRs were skipped before any of their runs; the Python process to load the fix for that lock, while one PR's summaries were being written and before any of its arms ran; and the Python process again to load the repaired recipe for azure-cli #32298. No agent run was cut short, and the skipped PRs were run after the restart. Several fixes were made before the runs they affect. Container recipes (not part of the plan) were fixed during pilot setup: Python dependencies resolved as of the PR's merge time with the project's own constraints, generated files that projects keep out of git (Home Assistant translations, zitadel's protobuf code) produced in every container, and OpenTofu installed for terragrunt. Amendment 2 limited Go stage 1 to the PR's changed test functions. The eligibility rule differs from the earlier sampled checks in two ways, both fixed before any agent run: it now needs a runnable test file, and

regex escapes that a copied script had lost were restored. Amendment 3, made after the pilot and before any main-run agent run, changed one line of the agent prompt so the file tools are given the checkout's own paths rather than /work paths (Section 4.7).

8. Reproducibility

The study lives in `studies/04-review-intent` of the public repository github.com/Shiningingy/ai-code-trust-studies. The frozen plan is `plan.md`; it was frozen in commit `21592ab` on 2026-11-07, before the pilot's first agent run, and its Amendments section lists every later change. Selection scripts and outputs are in `selection/` (`pool.py`, `overlap.py`, `langs.py`, `eligible.py`, `draw.py`; outputs `eligible.csv`, `main_walk.csv`, `agents.csv`). The pipeline is in `scripts/`, and `scripts/analyze.py` carries out Section 4.6. The seed for the draw, the order of arms and the bootstrap is 20261107. All model calls ran through `claude -p` on a Claude subscription, with model, flags, Claude Code version and usage logged per run. The AgentReviewChat parquet files come from Hugging Face `Suzhen/AgentReviewChat` and are not copied into the repository.

Appendix A. Audit of excluded PRs

Every exclusion was checked for whether it came from the PR itself or from this study's setup. A PR excluded because of the setup (a container recipe, a time limit) may be repaired and run. That is a correction, not a replacement, which the plan forbids. The same test applies to all 14 exclusions, and none of them had any agent run, so no result can bias which ones are repaired. Any repair is listed in the plan's Amendments.

PR	Check failed	Reason	Cause	Repairable
zeta-chain/node #4153, #4192, #4203, #4330	1	The merge commit's file set differs from the PR's files, so the base commit would not match the PR	PR history	No
getsentry/sentry-python #4873	4	No human-started review conversation to summarise (arm B impossible)	PR	No
pingcap/tidb #60776	4	No human-started review conversation to summarise (arm B impossible)	PR	No
Azure/azure-cli #32115	3	PR tests ran (171 passed on base and merge); none fails before the change	PR	No
ava-labs/avalanchego #4259	3	PR tests ran (96 pass on base and merge); none fails before the change	PR	No
pingcap/tidb #63878	3	PR tests ran (13 pass on base and merge); none fails before the change	PR	No
streamlit/streamlit #12074	3	PR tests ran (56 passed on base and merge); none fails before the change	PR	No
volcano-sh/volcano #4354	3	PR tests ran (24 pass on base and merge); none fails before the change	PR	No
streamlit/streamlit #11690	3	The only changed test is a browser end-to-end test with pixel-exact screenshots (Playwright); pytest collected no tests	Setup, in principle	Not attempted: needs browsers in the container, and pixel-exact screenshots are unlikely to match across machines
Azure/azure-sdk-for-python #40615	2, then 3	Tests replay recorded HTTP traffic through the Azure SDK test proxy; without the proxy every test failed. Repaired: the recordings now replay offline, but the tests that exercise the bug are live-test-only	Setup, then PR	Repaired; still excluded (no fail-to-pass test offline)
Azure/azure-sdk-for-python #41742	2	Tests need a live Cosmos DB account or emulator (the test setup connects on import)	Setup	Not attempted: needs a full database emulator inside the container

Azure/azure-cli #32298 passed the build checks, but its tests ran longer than the 30-minute stage-1 limit (a setup cause).

Outcome: #32298 was repaired by running the same tests in eight parallel workers (pytest-xdist); one pass then took about 6 minutes, and the PR yielded 9 fail-to-pass and 284 pass-to-pass tests and joined the run. #40615's recorded tests now replay offline through the test proxy (457 pass-to-pass, none flaky), but the only tests that exercise the bug are marked live-test-only and have no recordings, so it moved from check 2 to check 3 and stays excluded. Neither repair changed any other PR's environment.

Appendix B. Prompts

Agent prompt (same in every arm):

```
You are working in a checkout of the {repo} repository (the current directory). Resolve the
issue below by changing the code.

<issue>
{issue}
</issue>
{guidance: empty in arm A; the block below in arms B and C}
Notes:
- The `run` tool executes shell commands in the project's Linux environment, in this same
directory (inside that environment it appears as /work). Use it for tests, linters and
scripts. There is no network access.
- The file tools (Read, Edit, Write, Glob, Grep) take this directory's own paths, absolute or
relative. They cannot open /work paths: when a command's output names a file under /work, open
the same relative path here.
- Do not commit. Leave your changes in the working tree.
- Do not refer to this task or its instructions in code, comments or docs.
```

Guidance block (arms B and C):

```
<guidance>
Guidance on this change (what it should do and why):
{summary}
</guidance>
```

Summariser prompt (same for B and C; `{max_words}` is 150, and `{forbidden}` lists names the leak check found, on regeneration only):

```
Below are inline code-review comments left on a pull request in the repository {repo}. Each
comment comes with the file path and the end of the diff hunk it refers to. The pull request
resolves this issue:

<issue>
{issue}
</issue>

<comments>
{comments}
</comments>
```

```
Write guidance for a developer who will implement this change from scratch, without seeing the
pull request or these comments. Say what the comments ask the change to do, and why, as their
authors saw it. The developer has never seen any draft of this change, so state each point as
a requirement for the finished change, not as an edit to draft code. Rules:
- At most {max_words} words, plain sentences or short bullet points.
- What and why only, never how: no code, no snippets, no backticks, no names of functions,
variables, classes, files or settings that the pull request itself introduces.
- Do not mention comments, reviewers, review or the pull request. Write it as guidance about
the change itself.
- If the comments only raise minor points, summarise those briefly.{forbidden}

Reply with the guidance only.
```

Appendix C. A worked example

Drawn with seed `20261107-example` from the 56 complete PRs, leaving out the two spot-check PRs (`scripts/example.py`): **apache/superset #34763** (Python).

Issue given to every arm (shortened here):

> # 31813 Upload forms have non-actionable error message
>
> ### Bug description
>
> When trying to upload a CSV file to Superset, if there's a problem with the data such as a numeric column containing an empty value or a data column with wrong formatting, the modal shows `Database upload file failed` or `Table already exists` without any context about the error. This message does not help users to find what's the problem with their CSV file resulting in a very poor user experience and frequent on-call requests.
>
> The error should contain more information about what the problem was such as the column, row and type of problem.
>
> ### Screenshots/recordings
>
> *No response*
>
> ### Superset version
>
> master / latest-dev
>
> ### Python version
>
> 3.9

Node version

16

Browser

Chrome

Additional context

No response

Checklist

- [x] I have searched Superset docs and Slack and didn't find a solution to my problem.
- [x] I have searched the GitHub issue tracker and didn't find a similar bug report.
- [x] I have checked Superset's logs for errors and if I found a relevant Python stacktrace, I included it here as text in the "additional context" section.

Summary of human review (arm B, 147 words):

- Track the selected database as one piece of state holding the whole database object, not an id plus a separate parallel state. Duplicate state is unnecessary.
- Don't add frontend checks for particular engines, such as rejecting Trino table names that contain spaces. Instead, catch failures from the backend step that runs the table-creation statement and return them to the user as clear error messages. This general approach is preferred over checking for specific cases.
- Keep the maximum number of detailed errors shown, with a note on how many more exist, as a fixed value. Don't make it a new configuration option. The limit depends on how much space the UI has, so admins don't need to tune it.
- In the tests, set up the shared database-list mock before each test and reset it after each one, rather than repeating it in every test.

Summary of agent review (arm C, 143 words):

The CSV upload should report specific, actionable errors when data fails type conversion. The finished change needs to do the following:

- **Find all bad values in a column.** Don't stop at the first invalid value. Users should get a full picture of what's wrong with their file. The number of errors actually shown can still be capped.
- **Keep the display limit configurable.** The cap on how many errors are shown exists to avoid huge responses and a poor experience when a file has thousands of errors. Deployments that have already set their own limit through the existing configuration option must keep that behaviour, so don't replace it with a fixed value.

> - **Test the no-truncation case properly.** When the number of errors is under the limit, the test should confirm that no "and N more errors" style summary appears at all.

Rubric and Opus verdicts per arm (met / not_met / cant_tell):

Item	Behaviour	A	B	C
R1	When a CSV upload sets a column's data type explicitly and some values in that column can't be converted to it (e.g. 'abc' in an int column, 'one point five' in a float column), the upload fails with an error message that names the column and the target type, not a generic 'Database upload file failed'.	not_met	not_met	met
R2	The error message points to where each bad value is: it gives the offending value and the 1-based line number in the file. The line number accounts for the header line and for any configured header_row offset (with the default header, data row index 0 is line 2; with header_row=1 it is line 3).	not_met	not_met	met
R3	The error reports how many invalid values the column has in total, but lists only a capped number of them (the first 5). If there are more, it adds a summary such as '... and N more error(s)' so a large file doesn't produce a huge message.	not_met	not_met	not_met
R4	Empty or null cells are not counted as invalid values in the detailed report. Only non-null values that can't be converted are listed and counted.	not_met	not_met	met
R5	Valid explicit types still work. Integer and float columns (int32/int64/float32/float64, including integer values in a float column) convert to the requested dtype. If a type is given for a column that isn't in the file, it is ignored and no error is raised. Uploads with no explicit types keep pandas' normal type inference.	met	cant_tell	met
R6	The detailed message reaches the user. The generic upload exception handler doesn't swallow it or replace it with a generic message: it passes through the underlying exception's message (or its str()). Also, an explicit upload error raised inside the CSV read isn't re-wrapped by the reader's broader exception handlers.	met	not_met	not_met
R7	If the detailed analysis of invalid values itself fails, or no specific bad rows can be found, the error still names the column and target type and includes the original conversion error text instead of crashing.	not_met	not_met	not_met
R8	The 'table already exists' check is case-insensitive in the sense that a table stored under a lower-cased name also counts as existing, so the 'Table already exists' handling matches what the database actually stores.	not_met	not_met	not_met

Rubric score: A 0.25, B 0.00, C 0.50.

This PR is one of the 13 where B scored below A. The human-review summary is about frontend state, engine-specific checks and test set-up; the rubric is about the backend error message. The

two reviews also disagree on one point: human review asks for a fixed cap on listed errors, agent review for a configurable one. It illustrates how a review summary can steer the agent toward the reviewers' concerns rather than the issue's.